

ConcurrentChat: Core Functionality Implementation

Group Project Deliverable 2 (Saturday) - Option 2: Simple Chat Application

Languages: Rust and Go

Rahul Solanki, Krinal Soni, and Bijay Raj KC

University of the Cumberland

MSCS-632-M30: Advanced Programming Language

Jay Thom

October 3, 2026

ConcurrentChat: Core Functionality Implementation

Status

The core of ConcurrentChat is implemented and operational in both languages. Three simulated users run concurrently, each posting a join event, three messages and a leave event to a hub that owns the message history. Both implementations record the same fifteen messages and shut down cleanly. Filtering, keyword search, direct messages, the interactive prompt and the throughput benchmark are scheduled for Sunday and are not part of this deliverable.

Table 1

Requirement Status at the End of Day 2

Requirement	Status
Simulate multiple users sending messages concurrently	Complete
Store message history with timestamp and user ID	Complete
Hub solely owns history, so no lock is required	Complete
Clean shutdown reporting the recorded count	Complete
Filtering and search by user or keyword	Day 3
Direct messages, statistics, interactive prompt, benchmark	Day 3
Unit tests	Day 3

Implementation

Both programs follow the architecture agreed on Friday: user workers hold only a sending handle, and the history lives inside a single hub worker. Rust spawns one Tokio task per user and sends commands over an mpsc channel, with replies returning on a oneshot channel so that exactly one response is expressible in the type. Go spawns one goroutine per user and uses a select loop in the hub over its channels, with a sync.WaitGroup governing lifecycle.

The first substantive difference appeared immediately, in how the five message kinds are modelled. Rust expresses them as a data-carrying enum in which each variant holds exactly the fields it needs, and the compiler checks that every match covers every variant. Go has no such construct, so a message is one struct with a Kind tag plus the union of the fields any kind might

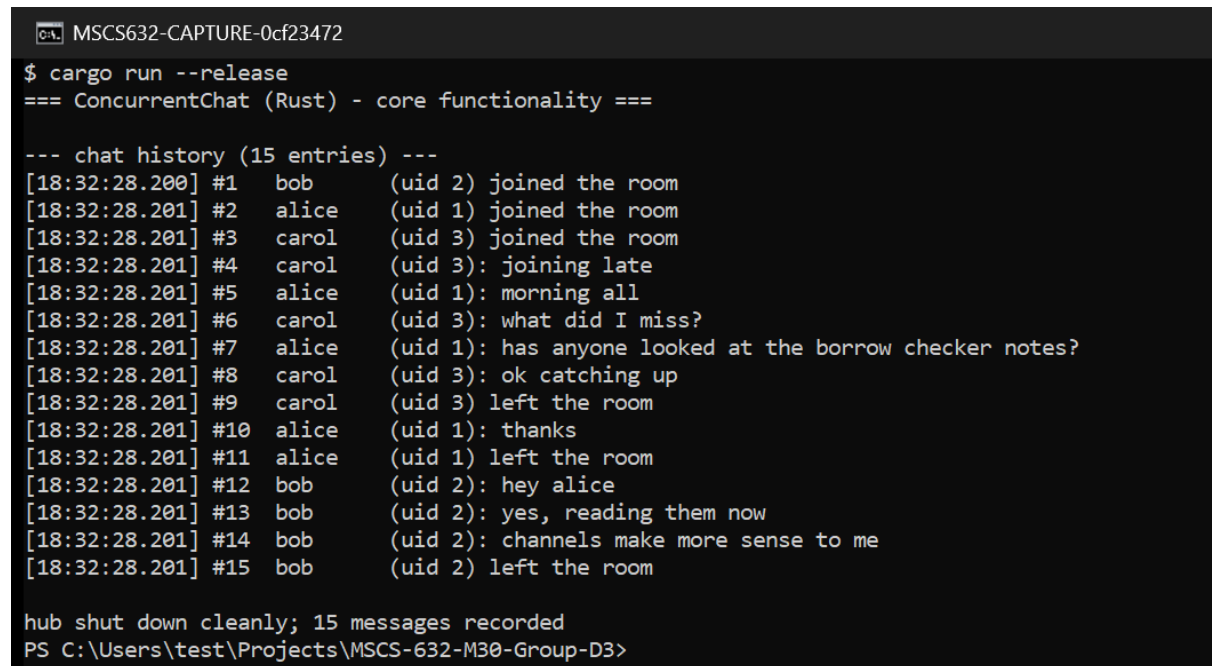
need. The Body field therefore exists on join and leave events, where it is meaningless, and a switch on Kind that misses a case compiles without complaint. Both programs include a fallthrough branch in their rendering function, but only the Go one can actually be reached.

Verification

Both implementations were built and run in WSL2 Ubuntu 24.04 using rustc and cargo 1.95.0 and Go 1.27.1, so that a single machine underlies every result. Each run records fifteen messages: three users times one join, three text messages and one leave. The ordering of the interleaved messages differs between runs and between the two languages, which is the concurrency being visible rather than a defect; the counts do not vary.

Figure 1

Rust Core Implementation Running



```

MSCS632-CAPTURE-0cf23472
$ cargo run --release
=== ConcurrentChat (Rust) - core functionality ===

--- chat history (15 entries) ---
[18:32:28.200] #1 bob (uid 2) joined the room
[18:32:28.201] #2 alice (uid 1) joined the room
[18:32:28.201] #3 carol (uid 3) joined the room
[18:32:28.201] #4 carol (uid 3): joining late
[18:32:28.201] #5 alice (uid 1): morning all
[18:32:28.201] #6 carol (uid 3): what did I miss?
[18:32:28.201] #7 alice (uid 1): has anyone looked at the borrow checker notes?
[18:32:28.201] #8 carol (uid 3): ok catching up
[18:32:28.201] #9 carol (uid 3) left the room
[18:32:28.201] #10 alice (uid 1): thanks
[18:32:28.201] #11 alice (uid 1) left the room
[18:32:28.201] #12 bob (uid 2): hey alice
[18:32:28.201] #13 bob (uid 2): yes, reading them now
[18:32:28.201] #14 bob (uid 2): channels make more sense to me
[18:32:28.201] #15 bob (uid 2) left the room

hub shut down cleanly; 15 messages recorded
PS C:\Users\test\Projects\MSCS-632-M30-Group-D3>

```

Note. Fifteen messages recorded. Message identifiers are assigned by the hub in arrival order, so the interleaving of alice, bob and carol reflects the scheduler rather than the script.

Figure 2*Go Core Implementation Running*

```

C:\ MSCS632-CAPTURE-af419021
$ ./chat
=== ConcurrentChat (Go) - core functionality ===

--- chat history (15 entries) ---
[18:32:15.185] #1 carol (uid 3) joined the room
[18:32:15.185] #2 bob (uid 2) joined the room
[18:32:15.185] #3 alice (uid 1) joined the room
[18:32:15.186] #4 bob (uid 2): hey alice
[18:32:15.186] #5 carol (uid 3): joining late
[18:32:15.186] #6 alice (uid 1): morning all
[18:32:15.187] #7 carol (uid 3): what did I miss?
[18:32:15.187] #8 bob (uid 2): yes, reading them now
[18:32:15.187] #9 alice (uid 1): has anyone looked at the borrow checker notes?
[18:32:15.188] #10 bob (uid 2): channels make more sense to me
[18:32:15.188] #11 bob (uid 2) left the room
[18:32:15.188] #12 carol (uid 3): ok catching up
[18:32:15.188] #13 carol (uid 3) left the room
[18:32:15.188] #14 alice (uid 1): thanks
[18:32:15.188] #15 alice (uid 1) left the room

hub shut down cleanly; 15 messages recorded
PS C:\Users\test\Projects\MSCS-632-M30-Group-D3>

```

Note. The same fifteen messages, with a different interleaving produced by the Go scheduler.

Code Quality and Testing

Both files are organised the same way, with the domain model first, then the hub, then the driver, and each language-specific decision is commented where it occurs rather than only in this report. Testing at this stage was manual: repeated runs confirming that the recorded count is always fifteen, that every user contributes a join and a leave, and that both programs terminate without hanging. Automated unit tests, go vet and the Go race detector are part of Sunday's deliverable.

Issues Found and Resolved

Two problems surfaced during the day. First, an early Go draft gave the hub a separate channel for each command type. Because select chooses at random among ready cases, a query could be served before queued posts had been appended. The risk had been anticipated on Friday, and the fix is scheduled for Day 3: route every command through one ordered channel,

which is what the Rust design does implicitly by carrying a single command enum on one channel. Second, both implementations initially produced all of one user's messages before any of another's, because each worker ran to completion before yielding. Inserting an explicit yield between sends restored the interleaving that makes the concurrency observable.

Repository

The code, the README with build and run instructions, and these screenshots are in the repository at <https://github.com/iknalos/MSCS-632-M30-Group-D2>. The README documents the layout, the exact toolchain versions, the commands to build and run each implementation, and the language-specific features demonstrated in each file.

References

Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.

Klabnik, S., & Nichols, C. (2023). *The Rust programming language* (2nd ed.). No Starch Press.